Temporal Databases

Group Name: Query Stack

Name	Roll Number
Ashok Chandra R	23CS10059
Monica Reddy N	23CS10046
Rachamsetty Harsha Vardhan	23CS10056
Miryala Avinash Kumar	23CS10045

Abstract

High-frequency financial markets generate massive, continuous streams of tick data, creating severe bottlenecks for real-time database ingestion, analytical querying, and long-term storage. This project explores the adaptation of PostgreSQL into a high-performance temporal database capable of managing these demanding workloads. We designed and evaluated a comprehensive data pipeline, systematically benchmarking multiple ingestion strategies, schema architectures, and storage optimizations. Our results demonstrate that while naive row-wise insertion fails under real-time load, vectorized batch ingestion utilizing PostgreSQL's COPY protocol successfully sustains high-velocity throughput. Furthermore, a schema ablation study proves that declarative time-based partitioning combined with composite indexing reduces range-query latency by orders of magnitude. To optimize analytical queries, we implemented a continuous background aggregation pipeline that rolls up raw ticks into 1-second and 1-minute OHLCV bars. Finally, to mitigate the unbounded storage bloat of tick data, we conducted a byte-level memory analysis of LZ4 compression on batched TSV payloads. By evaluating cumulative storage growth and per-batch compression ratios, we demonstrate how lightweight compression algorithms can significantly reduce the storage footprint of time-series financial data without sacrificing ingestion speed. Ultimately, this prototype confirms that with rigorous architectural tuning, standard relational databases can effectively scale to serve as robust quantitative financial engines.

Introduction

Stock markets generate large amounts of trade data continuously. Every buy or sell transaction produces a record called a tick, which captures the stock symbol, price, volume, and timestamp of the trade. With multiple stocks trading at high frequency, thousands of ticks are produced every second, and this data must be stored and queried efficiently. Writing tick data fast enough to keep up with the market is a challenge on its own. Reading it back for analysis is another — querying millions of raw records for

something as simple as a daily price summary is slow and wasteful. Storage also becomes a concern as the dataset grows without bound over time. This project builds a prototype system in PostgreSQL to address these challenges. We implement and compare multiple ingestion strategies, apply partitioning and indexing to improve query performance, use an aggregation pipeline to convert raw ticks into OHLCV bars at one-second and one-minute resolutions, and explore compression techniques to reduce storage overhead. Each optimization is measured and compared to understand its actual impact on performance.

Tech Stack

- **PostgreSQL** — Primary database engine
- **Python** — Data generation, ingestion logic, and pipeline orchestration
- **NumPy / Pandas** — Data processing and serialization
- **psycopg2** — PostgreSQL adapter for Python
- **Alpaca Markets API** — Real historical tick data source
- **Streamlit** — Interactive frontend dashboard
- **Plotly** — Financial charting and visualizations

Objective

The goal of this project is to design and evaluate a PostgreSQL-based prototype capable of handling high-frequency financial tick data. Specifically, the project aims to:

- Benchmark multiple ingestion strategies to identify which can sustain real-time tick throughput.
- Evaluate the impact of schema design choices — including partitioning and indexing — on both write and read performance.
- Build a hierarchical aggregation pipeline that converts raw ticks into OHLCV bars at one-second and one-minute resolutions.
- Explore array-based compression using PostgreSQL's LZ4 storage as an alternative to row-per-tick storage.
- Provide a live dashboard for visualization and benchmarking.

Methodology

System Overview

The system is built as a pipeline with four main components: data generation, ingestion, aggregation, and a frontend dashboard. Tick data enters the system either as synthetically generated batches or as real historical data fetched from the Alpaca API. It is then written into PostgreSQL using one of several ingestion strategies. A background aggregation process continuously rolls up raw ticks into OHLCV bars. The Streamlit dashboard reads from the database and provides live charts and benchmark comparisons.

Database Schema Design

Motivation

As tick data accumulates continuously, storing everything in a single large table becomes impractical. Query performance degrades as the table grows, and bulk deletions of old data become expensive. A structured schema design is therefore essential before any data enters the system.

ER Diagram

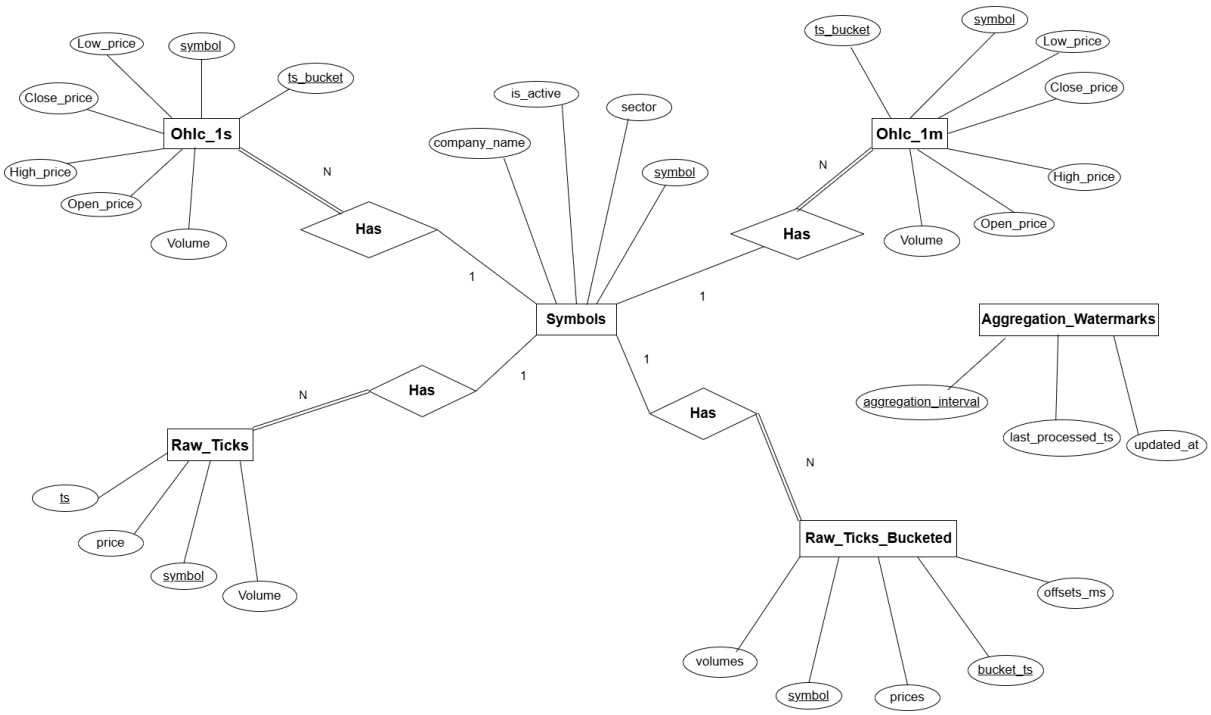

Figure 1: Entity-Relationship Diagram of the Temporal Database Schema

Data Definition Dictionary

The database schema is organized into six primary entities designed to handle high-frequency temporal data. The following data dictionary defines the attributes, inferred data types, and structural purpose of each entity within the architecture.

1. Symbols (Dimension Table)

Acts as the central dimension table, storing static metadata for the financial assets being tracked.

Attribute Name	Data Type	Key	Description
symbol	VARCHAR(10)	PK	Unique ticker symbol for the asset (e.g., 'AAPL', 'TSLA').
company_name	VARCHAR(100)	-	Full registered name of the company.
sector	VARCHAR(50)	-	The market sector to which the company belongs.
is_active	BOOLEAN	-	Flag indicating if the system is actively tracking this asset.

2. Raw_Ticks

Stores the raw, unaggregated trade executions exactly as they arrive from the data source.

Attribute Name	Data Type	Key	Description
ts	TIMESTAMP	-	The exact timestamp of the trade execution.
symbol	VARCHAR(10)	FK	Reference to the Symbols table.
price	NUMERIC	-	The price at which the asset was traded.
Volume	INTEGER	-	The number of shares exchanged in this single transaction.

3. Raw_Ticks_Bucketed

An optimized, compressed storage structure that groups high-frequency ticks into array blocks to drastically reduce storage footprint.

Attribute Name	Data Type	Key	Description
bucket_ts	TIMESTAMP	PK	The truncated timestamp marking the start of the bucket.
symbol	VARCHAR(10)	FK	Reference to the Symbols table.
prices	REAL[]	-	LZ4-compressed array of all trade prices within this bucket.
volumes	INTEGER[]	-	LZ4-compressed array of all trade volumes within this bucket.
offsets_ms	INTEGER[]	-	Array of millisecond offsets from bucket_ts for precise timing.

4. Ohlc_1s

Pre-aggregated candlestick data representing one-second trading intervals.

Attribute Name	Data Type	Key	Description
ts_bucket	TIMESTAMP	PK	The 1-second interval timestamp.
symbol	VARCHAR(10)	PK, FK	Reference to the Symbols table.
Open_price	NUMERIC	-	The first traded price within this 1-second window.
High_price	NUMERIC	-	The highest traded price within this 1-second window.
Low_price	NUMERIC	-	The lowest traded price within this 1-second window.
Close_price	NUMERIC	-	The final traded price within this 1-second window.
Volume	INTEGER	-	Total cumulative volume traded during this second.

5. Ohlc_1m

Pre-aggregated candlestick data representing one-minute trading intervals.

Attribute Name	Data Type	Key	Description
ts_bucket	TIMESTAMP	PK	The 1-minute interval timestamp.
symbol	VARCHAR(10)	PK, FK	Reference to the Symbols table.
Open_price	NUMERIC	-	The first traded price within this 1-minute window.
High_price	NUMERIC	-	The highest traded price within this 1-minute window.
Low_price	NUMERIC	-	The lowest traded price within this 1-minute window.
Close_price	NUMERIC	-	The final traded price within this 1-minute window.
Volume	INTEGER	-	Total cumulative volume traded during this minute.

6. Aggregation_Watermarks

A state-management table used by background workers to track the progress of continuous aggregation pipelines.

Attribute Name	Data Type	Key	Description
`aggregation_interval`	VARCHAR(50)	PK	Identifier for the specific pipeline (e.g., '1s_rollup').
`last_processed_ts`	TIMESTAMP	-	The highest timestamp safely aggregated by the worker.
`updated_at`	TIMESTAMP	-	System timestamp of when this watermark was last modified.

Frontend Dashboard

The frontend is built using Streamlit and Plotly. It provides two main tabs.

The **Market Analysis** tab allows users to select a stock symbol and time range, choose between one-second and one-minute resolution, and view candlestick charts with a 20-period SMA overlay, volume bars, and key metrics including latest price, total volume, volatility, and moving average.

The **Optimization Benchmarks** tab runs live query comparisons between raw tick scans and pre-aggregated queries, displaying latency and the speedup factor. It also shows a storage comparison between the raw ticks table and the aggregated OHLCV table.

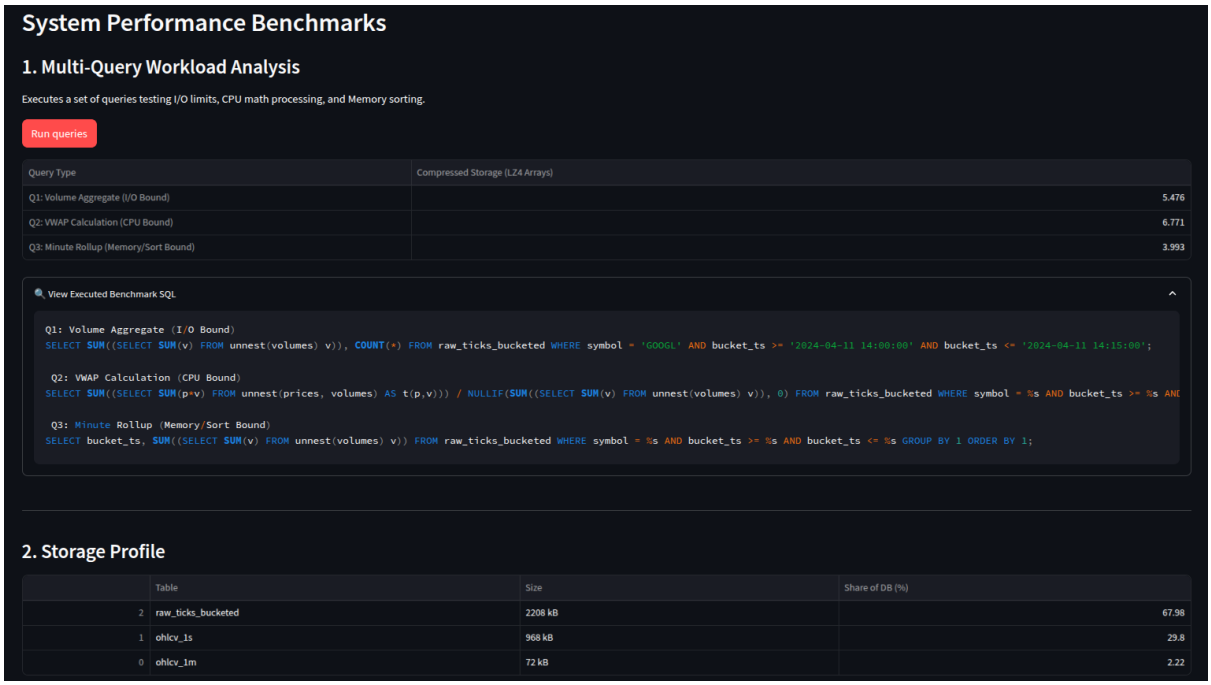

Figure 2: Streamlit Dashboard — Market Analysis Tab: Candlestick chart with 20-SMA overlay, volume bars, and strategic performance indicators for GOOGL

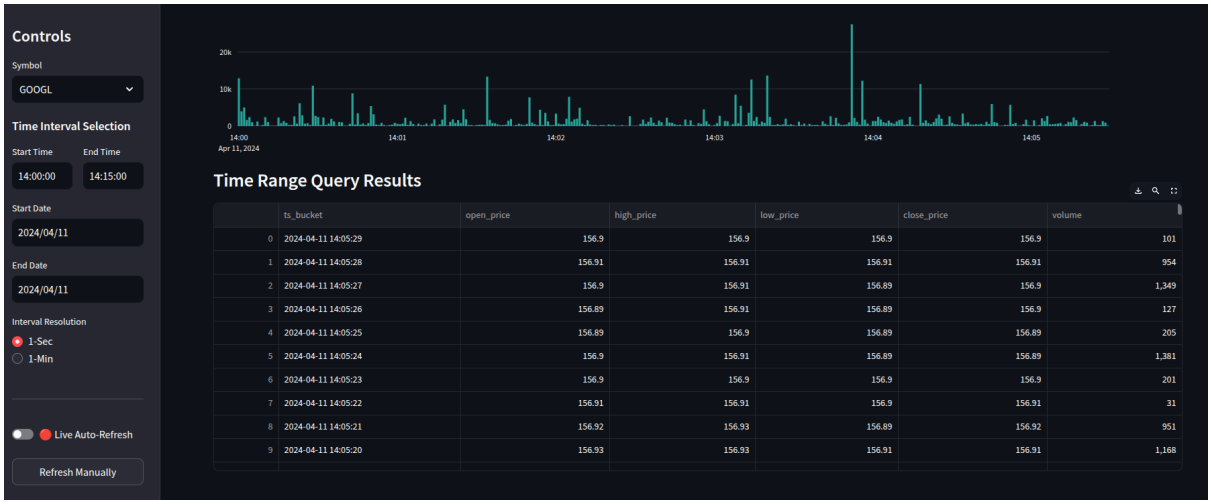

Figure 3: Streamlit Dashboard — Time Range Query Results: OHLCV tabular output with candlestick chart for a selected time interval

Figure 4: Streamlit Dashboard — Optimization Benchmarks Tab: Detailed execution metrics comparing LZ4 Arrays vs Aggregated queries, with storage breakdown by table

Data Generation

Two data sources were implemented to feed the ingestion pipeline.

Mock Generator (`mock_gen.py`) generates fully synthetic tick data using NumPy. It randomly assigns stock symbols, prices in the range \$100–\$1500, volumes, and timestamps at a configurable rate. The speed multiplier N controls how many virtual seconds of data are generated per real second, allowing stress testing at different throughput levels.

Historic Generator (`live_gen.py`) fetches real historical trade data from the Alpaca API. It applies a two-year time offset to retrieve actual past trades and replays them second by second, yielding one batch per second to simulate a live stream. Ten symbols

are tracked: GOOGL, META, TSLA, NVDA, AMZN, NFLX, MSFT, AAPL, TSMC, and INTC.

Ingestion Strategies

Four ingestion strategies were implemented and benchmarked against each other. All strategies disable synchronous commit (`SET synchronous_commit = OFF`) to improve write throughput, and all dynamically create daily partitions as needed.

1. Row-Wise INSERT

The simplest approach. Each tick is inserted individually using a standard `INSERT` statement inside a loop. This incurs one round-trip to the database per tick, making it extremely slow at high tick rates. It serves as the baseline to demonstrate why naive insertion does not scale.

2. Batch List (execute_values)

Uses `psycopg2.extras.execute_values` to send an entire batch of ticks in a single SQL statement. This dramatically reduces the number of round-trips to the database. The batch is constructed as a list of tuples and sent in one call.

3. Batch COPY via NumPy/StringIO

Uses PostgreSQL's `COPY` protocol, which is the fastest way to load bulk data into a table. The batch is converted to a Pandas DataFrame, serialized into a tab-separated in-memory buffer using `StringIO`, and streamed directly into the partition using `cursor.copy_from`. This avoids SQL parsing overhead entirely.

4. Compressed Array Bucketing

An alternative storage strategy that fundamentally changes the schema. Instead of inserting one row per tick, ticks within the same second and symbol are grouped together. Prices, volumes, and millisecond offsets are stored as PostgreSQL arrays in a single row per (symbol, second) bucket. LZ4 compression is applied at the column level on these arrays. This reduces row count significantly and changes the storage footprint of raw tick data.

Aggregation Pipeline

Motivation

Querying raw tick data directly for analytical purposes — such as computing a price trend over the last hour — requires scanning millions of rows. Pre-aggregating tick data into OHLCV bars at different time resolutions allows such queries to run over a much smaller dataset.

Pipeline Design

The aggregation pipeline runs as a separate background process (`genaggregate.py`). It uses a watermark stored in `Aggregation_Watermarks` to track the last processed timestamp, advancing one second at a time.

Every second, it queries all raw ticks within that one-second window, computes the open (first price), high (max), low (min), close (last price), and total volume per symbol, and

inserts the result into `OHLCV_1s`. When a full minute of one-second bars is complete, it rolls them up into a single row in `OHLCV_1m` using the same OHLCV logic over the minute window.

Gap Filling

If no ticks arrive for a particular symbol in a given second (which can happen during low-activity periods), the pipeline fills the gap by repeating the last known close price with zero volume. This ensures `OHLCV_1s` is always complete and continuous for all symbols.

Query Types

The system supports the following analytical query types, all implemented in the Streamlit dashboard:

1. **Time Range Query:** Retrieves OHLCV data between two user-specified timestamps.
2. **OHLCV Aggregation:** Renders candlestick charts from aggregated price bars.
3. **Moving Average:** Computes a 20-period simple moving average over close prices.
4. **Volume Analysis:** Calculates total traded volume over a selected time range.
5. **Latest Price:** Fetches the most recent close price for a selected symbol.
6. **Price Volatility:** Measures standard deviation of close prices over a time interval.

Results

Ingestion Speed: Row-Wise vs Batching

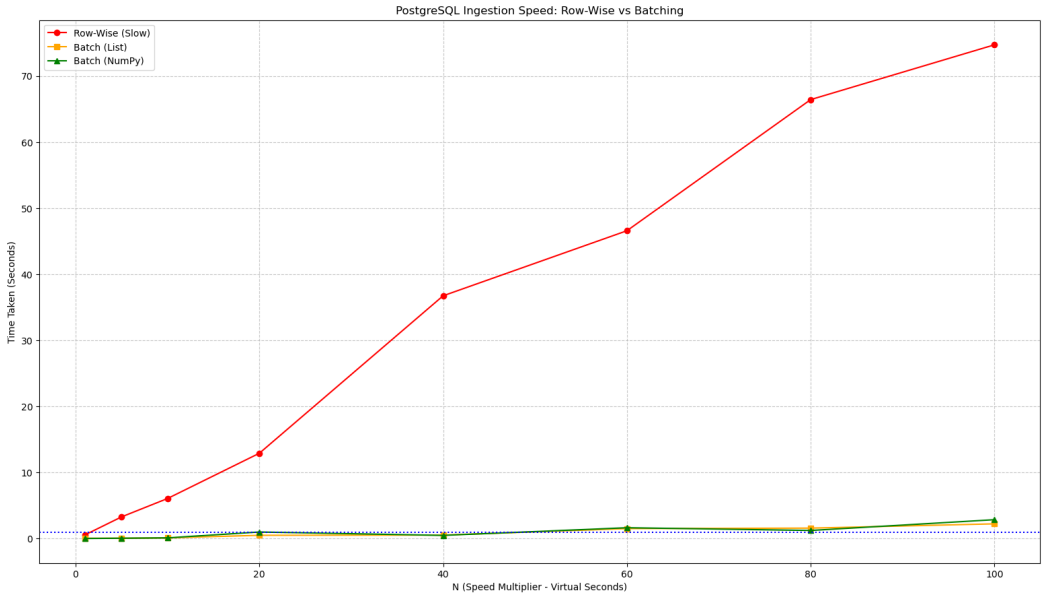

Figure 5: PostgreSQL Ingestion Speed: Row-Wise vs Batch Methods

Row-wise insertion degrades rapidly and becomes unusable beyond small tick rates, while both batch methods remain well under the 1-second real-time limit across all tested multipliers. Batching is a requirement, not merely an improvement, for real-time tick ingestion.

Schema Comparison: Write and Query Latency

Figure 6: DB Write Latency, End-to-End Latency, and Query Latency across Schema A, B, and C (over 230+ batches and 90+ queries)

Schema B (partitioned) achieves the lowest write latency at 30.07 ms vs 44.50 ms for Schema A. The key differentiator is query latency: Schema C (partitioned + indexed) achieves 5.63 ms vs 16.55 ms for Schema B and 26.98 ms for Schema A, with the composite index on `(symbol, ts)` enabling direct row lookup within partitions.

Query Throughput

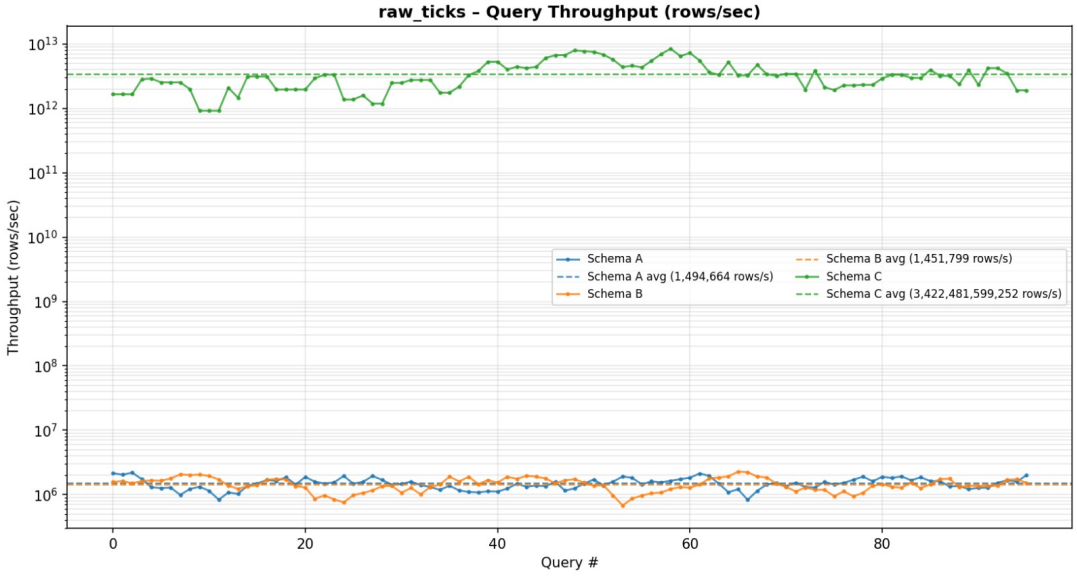

Figure 7: Query Throughput Comparison across Schema A, B, and C (rows/sec, log scale)

Schema C sustains throughput around 10^{13} rows/sec versus 10^{6} rows/sec for Schema A and B — a seven-order-of-magnitude difference. The composite index allows PostgreSQL to resolve range queries through the index structure alone, bypassing full data-page reads.

Query Stack Architecture vs Naive Baseline

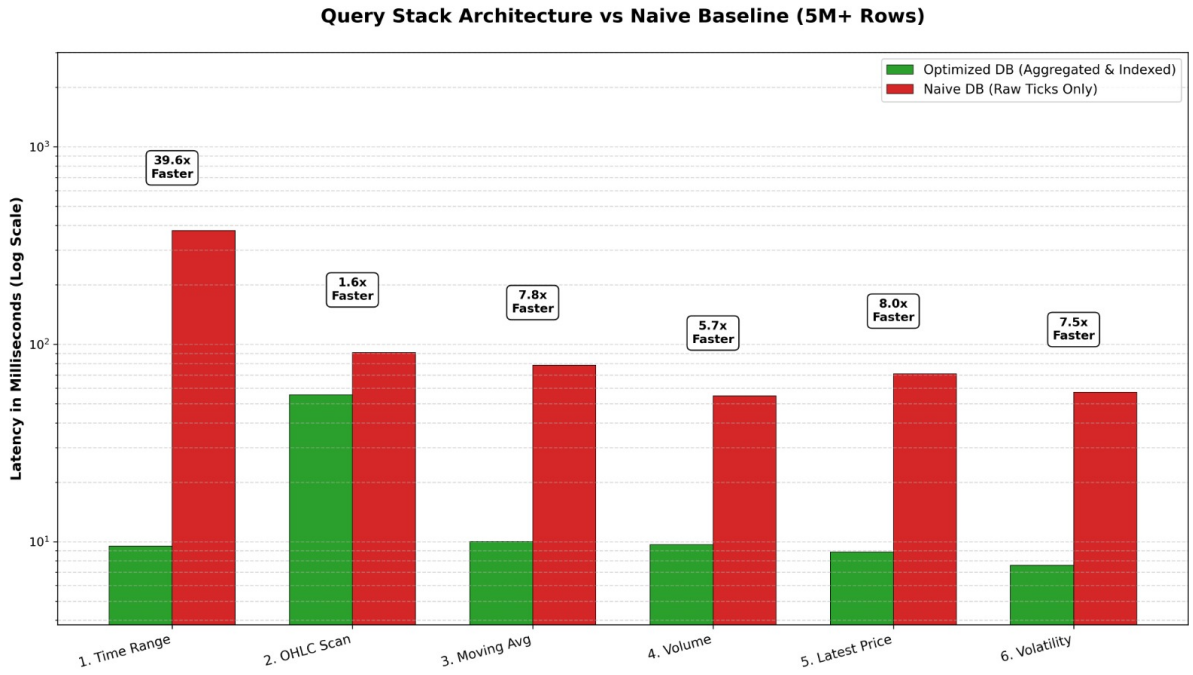

Figure 8: Query Stack Architecture vs Naive Baseline: Latency comparison (log scale, ms) across six analytical query types on a 5M+ row dataset.

On a 5M+ row dataset, the optimized architecture delivers consistent speedups across all query types: Time Range (**39.6**×), Latest Price (**8.0**×), Moving Average (**7.8**×), Price Volatility (**7.5**×), Volume Analysis (**5.7**×), and OHLCV Scan (**1.6**×). These gains confirm that partitioning, indexing, and pre-aggregation deliver compounding benefits for real-time quantitative workloads.

Aggregation Query Latency

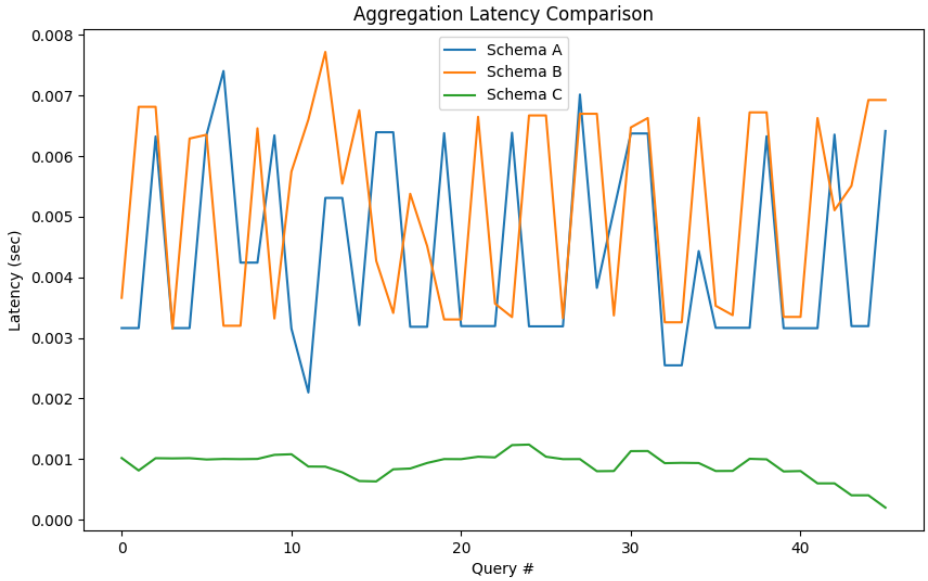

Figure 9: Aggregation Query Latency Comparison across Schema Variants

Schema C consistently maintains aggregation latency around 0.001 seconds, while Schema A and B fluctuate between 0.003 and 0.008 seconds, as the index eliminates full partition scans during `GROUP BY` operations.

Storage: LZ4 Compression Analysis

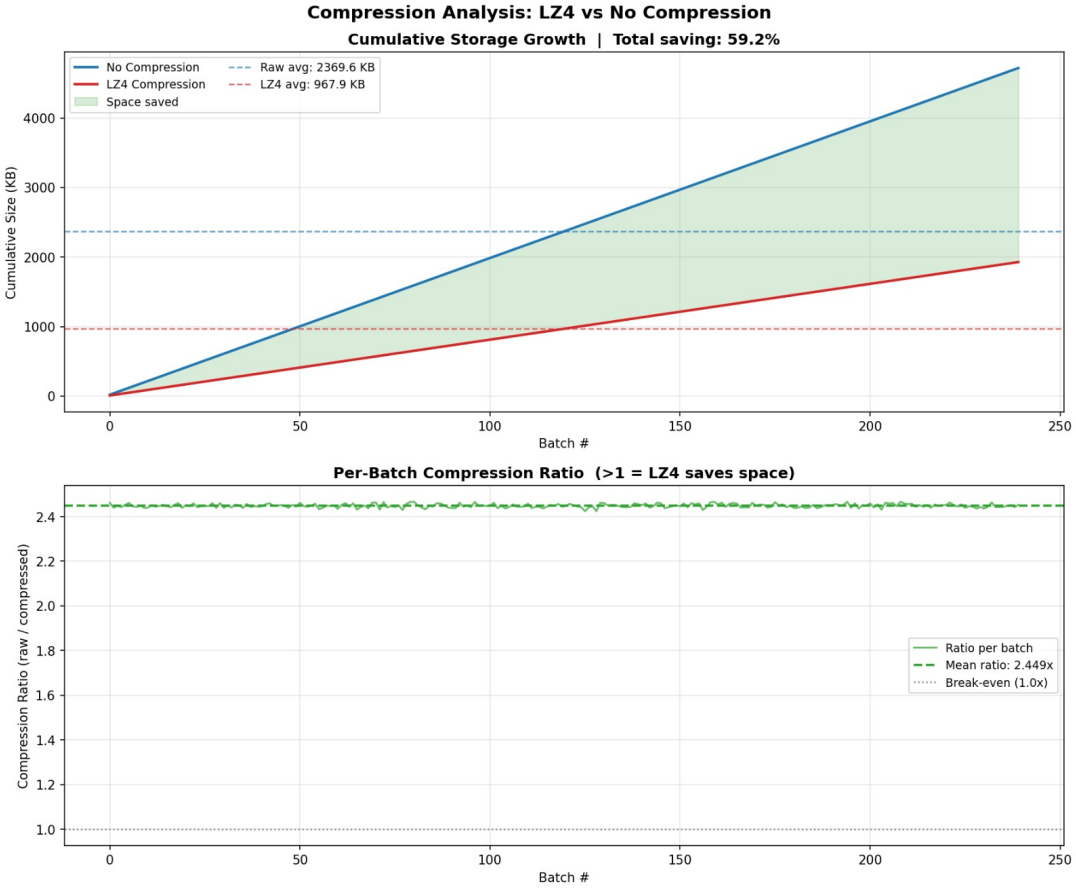

Figure 10: LZ4 Compression Analysis on Batched TSV Payloads: Cumulative storage growth and per-batch compression ratio over time

LZ4 compression achieves an average saving of **59.2%** per batch on real market tick data. Financial ticks exhibit strong temporal locality — gradual price changes and repeated volume values — which LZ4 exploits efficiently. The compressed storage curve diverges progressively from the uncompressed baseline, confirming LZ4 array compression as a scalable, low-overhead strategy for long-term tick data storage.

References

1. PostgreSQL Documentation. *Table Partitioning*. `https://www.postgresql.org/docs/current/ddl-partitioning.html`
2. PostgreSQL Documentation. *COPY Command*. `https://www.postgresql.org/docs/current/sql-copy.html`
3. Alpaca Markets API Documentation. `https://docs.alpaca.markets/`
4. Streamlit Documentation. `https://docs.streamlit.io/`
5. Plotly Documentation. `https://plotly.com/python/`